

CYBERSECURITY INTERNSHIP REPORT

Week 2 – Implementing Security Measures

Name: Abdul Ahad

Internship: Cybersecurity Analyst Internship

Week: 2 – Security Implementation

Objective

The objective of Week 2 was to improve the security of the web application by implementing essential security mechanisms including input validation, password hashing, token-based authentication, and secure HTTP headers.

1. Installation of Security Dependencies

The required security libraries were installed in the backend project.

Installed Packages:

- validator
- bcrypt
- jsonwebtoken
- helmet

2. Input Validation & Sanitization

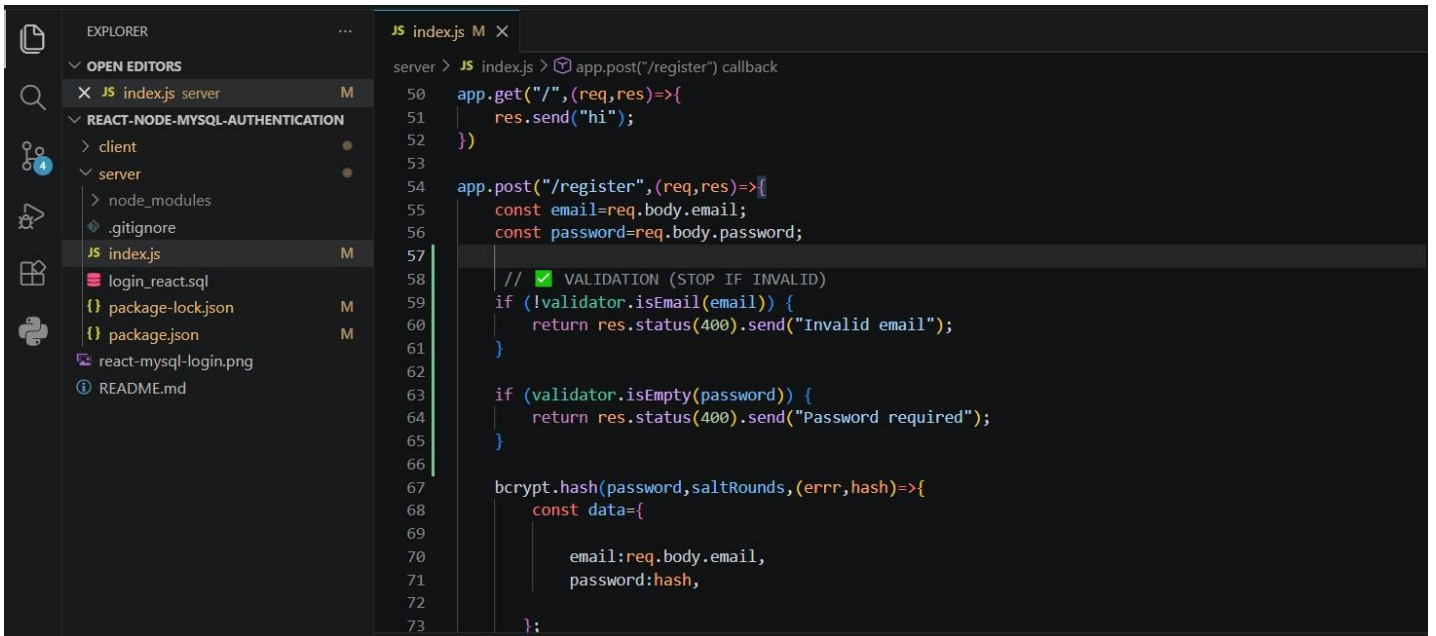
User inputs were validated using the `validator` library to ensure only properly formatted data is processed by the backend.

Implementation:

- Email format validation applied
- Empty password validation applied

```
if (!validator.isEmail(email)) {  
    return res.status(400).send("Invalid email");  
}  
  
if (validator.isEmpty(password)) {  
    return res.status(400).send("Password required");  
}
```

Validator implementation in backend code



```
server > JS index.js > app.post("/register") callback
50 app.get("/",(req,res)=>{
51   res.send("hi");
52 })
53
54 app.post("/register",(req,res)=>{
55   const email=req.body.email;
56   const password=req.body.password;
57
58   // ✅ VALIDATION (STOP IF INVALID)
59   if (!validator.isEmail(email)) {
60     return res.status(400).send("Invalid email");
61   }
62
63   if (validator.isEmpty(password)) {
64     return res.status(400).send("Password required");
65   }
66
67   bcrypt.hash(password,saltRounds,(errr,hash)=>{
68     const data={
69
70       email:req.body.email,
71       password:hash,
72
73     };
```

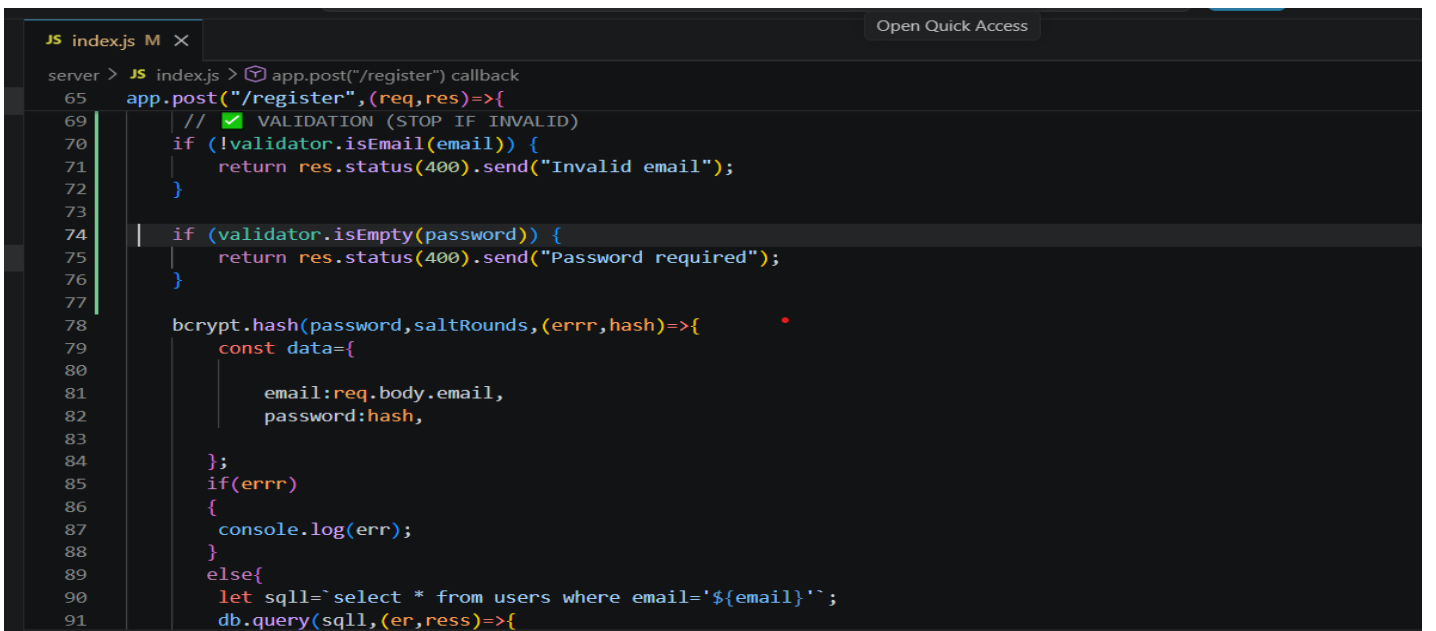
3. Password Security (bcrypt Hashing)

Password hashing was implemented using bcrypt to ensure secure storage of user credentials in the database.

Key Features:

- Passwords are hashed before storage
- Salt rounds used for added security

bcrypt password hashing in register route



```
server > JS index.js > app.post("/register") callback
65 app.post("/register",(req,res)=>{
66   // ✅ VALIDATION (STOP IF INVALID)
67   if (!validator.isEmail(email)) {
68     return res.status(400).send("Invalid email");
69   }
70
71   if (validator.isEmpty(password)) {
72     return res.status(400).send("Password required");
73   }
74
75   bcrypt.hash(password,saltRounds,(errr,hash)=>{
76     const data={
77
78       email:req.body.email,
79       password:hash,
80
81     };
82     if(errr)
83     {
84       console.log(err);
85     }
86     else{
87       let sql=`select * from users where email='${email}'`;
88       db.query(sql,(er,ress)=>{
```

4. Token-Based Authentication (JWT)

JWT (JSON Web Token) authentication was implemented to improve login security.

Implementation:

- Token generated after successful login
- Token includes user ID and email
- Token expires after a fixed time

```
const token = jwt.sign(  
  { id: result[0].id, email: result[0].email },  
  "secretKey",  
  { expiresIn: "1h" }  
);
```

JWT token generation code in index.js

```
JS index.js M X  
server > JS index.js > app.post("/login") callback > db.query() callback > bcrypt.compare() callback  
125 app.post("/login", (req, res) => {  
131   db.query(sql, (err, result) => {  
136     if (result.length > 0) {  
137         
138       bcrypt.compare(password, result[0].password, (errr, response) => {  
139           
140         // PASSWORD MATCHED  
141         if (response) {  
142             
143           req.session.user = result;  
144             
145           // JWT TOKEN  
146           const jwt = require("jsonwebtoken");  
147             
148           const token = jwt.sign(  
149             { id: result[0].id, email: result[0].email },  
150             "secretKey",  
151             { expiresIn: "1h" }  
152           );  
153             
154           logger.info(`Successful login: ${email}`); //Added this line  
155             
156           return res.send({  
157             login: true,
```

5. HTTP Security Headers (Helmet.js)

Helmet middleware was integrated to secure HTTP headers and protect the application from common web vulnerabilities.

Security Improvements:

- Prevents clickjacking attacks
- Disables MIME-type sniffing
- Adds secure HTTP headers

```
app.use(helmet());
```

Helmet middleware added in backend server file

A screenshot of a code editor window titled 'JS index.js M X'. The editor shows a JavaScript file named 'index.js' with the following code:

```
server > JS index.js > ...
22  const app=express();
23
24  const logger = winston.createLogger({
25    level: "info",
26    transports: [
27      new winston.transports.Console(),
28      new winston.transports.File({ filename: "security.log" })
29    ]
30  });
31
32  app.use(cors({
33    origin:"http://localhost:3000",
34    credentials:true,
35  }));
36  app.use(helmet()); //Added this line
37  app.use(express.json());
38  app.use(cookieparser());
39  app.use(bodyParser.urlencoded({extended:true}));
40  app.use(session({
41    key:"userId",
42    secret:"atanu",
43    resave:false,
44    saveUninitialized:false,
45    // cookie:{
```

6. Testing & Verification

After implementing security measures, the application was tested to verify proper functionality.

Observations:

- Login and registration working correctly
- Passwords stored in hashed format
- JWT token generated successfully
- Input validation blocking invalid entries

- Security headers applied successfully

Summary of Security Improvements

Implemented Enhancements:

- Input validation using validator
- Secure password hashing using bcrypt
- JWT authentication system
- HTTP security headers using Helmet